GeoSemTwin-X

A complete beginner's guide to the project, the prototype and the results

Geometry-grounded semantic SLAM → persistent, versioned digital twin of a warehouse

Prepared for: Gayathri K S · TECHgium 10th Edition, Presentation Round
Covers: every concept used, every module implemented, every experiment run, what each number means, and how to present and defend the work.

How to read this guide

Chapter 1 gives the big picture in plain words. Chapter 2 explains the vocabulary you will hear from judges. Chapter 3 walks through the system one block at a time (what goes in, what happens, what comes out, what we built). Chapter 4 is the story of how the prototype was built, including the problems we hit and fixed. Chapter 5 explains every experiment and result. Chapter 6 maps the code. Chapter 7 tells you how to present, what to say, and how to answer likely questions. Green boxes contain sentences you can say almost word for word.

Contents

1. The big picture: what GeoSemTwin-X is and why it matters
2. Concepts you must understand (glossary with intuition)
3. The system, block by block
4. What was actually done: the implementation journey
5. Experiments and results: how to read every number
6. Code map and how to run everything
7. How to present: script, demo, key numbers, judge Q&A
8. Appendix: honest limitations and how to talk about them

The project in one paragraph

A robot (or a person carrying a camera) walks through a warehouse. From the camera's colour images and depth images, GeoSemTwin-X works out **where the camera is** at every moment (SLAM), **what objects it sees** (forklifts, pallets, boxes, pallet trucks) using AI models that understand text prompts, and **where those objects are in 3D**. It then merges all of these sightings into a single, clean list of physical assets, each with one permanent ID, a class, a confidence and a 3D box. That list, together with a 3D map, is the **digital twin**. When the robot comes back later, the twin is updated as a new version, and the system reports which assets moved, appeared or disappeared, while ignoring false alarms caused by sensor noise. Finally, the system suggests where the robot should go next to learn the most (next-best view).

2.9 cm
camera-position error on the TUM benchmark

0.11
duplicates per asset (vs 10.6 per-frame, 0.29 centroid)

1.00
change-detection precision with verification

99.3%
map points within 5 cm of the true surface

1. The big picture

1.1 What is a digital twin?

A digital twin is a computer model of a real place that stays in sync with it. For a warehouse, the useful twin is not a pretty 3D picture; it is a **registry of assets**: "forklift A10 is at (x, y, z), last seen at 16:43, confidence 0.94". Managers can ask questions like "which pallet trucks are within 5 m of aisle 3?" or "what changed since yesterday?".

1.2 Why is this hard?

- **The camera does not know where it is.** Without GPS indoors, the system must estimate its own position from the images. Small errors add up over time ("drift").
- **AI detectors work on single images.** A detector looks at one frame and says "forklift here". In the next frame it says "forklift here" again. Without extra work, the same forklift is counted hundreds of times. In our synthetic test, plain per-frame labelling reported **289 objects for 22 real assets**.
- **Drift breaks naive merging.** If position estimates are slightly wrong, two sightings of the same forklift land in different places and look like two forklifts. Or a still object looks as if it moved.
- **Real change vs noise.** An object can "disappear" simply because the camera did not look at it this time. Reporting that as "removed" would be a false alarm.

1.3 The core idea of GeoSemTwin-X

Key insight

Let **geometry** decide which sightings belong to the same physical object (3D overlap, centre distance, shape), and let **semantics** (the AI label) only name it. On top of that, track **uncertainty** everywhere: how sure we are about the camera position, the depth, the label, and whether a region was actually observed. This is what "geometry-grounded" and "uncertainty-aware" mean in the title.

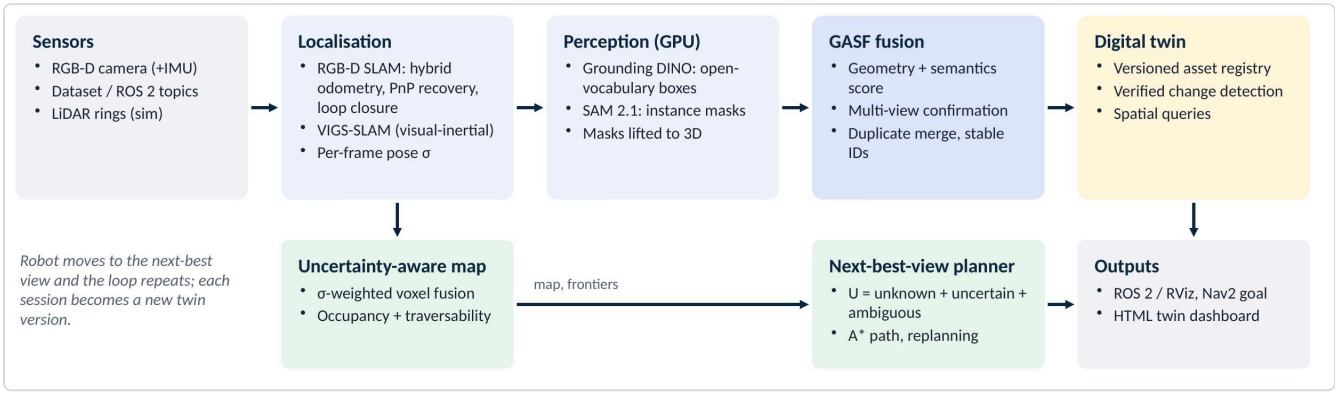


Figure 1. The GeoSemTwin-X pipeline (same diagram as slide 3). Data flows left to right; the map feeds the next-best-view planner, which tells the robot where to go next.

1.4 The pipeline in plain words

1. **Sensors.** An RGB-D camera gives a colour image and a depth image (distance for every pixel), 20–30 times per second. An IMU (accelerometer + gyroscope) measures motion.
2. **Localisation (SLAM).** Compares each new frame with earlier ones to work out how the camera moved. Also estimates how uncertain each step is.
3. **Perception.** Grounding DINO draws boxes around objects named by text ("forklift", "pallet"); SAM 2.1 turns each box into an exact pixel mask.
4. **3D lifting.** Using depth and the camera position, each mask becomes a small 3D point cloud in world coordinates.
5. **GASF fusion.** Each new 3D observation is matched to an existing asset or starts a new candidate. Candidates must be seen at least twice from at least two views before they become real assets. Fragments of the same asset are merged.
6. **Mapping.** Depth from every frame is fused into a 3D map, giving more weight to reliable measurements. An occupancy grid records which space is free, occupied or unknown.
7. **Digital twin.** Assets + map are saved as a version. A later session is compared with the previous version to find verified changes.
8. **Next-best view.** The planner scores candidate positions by how much unknown, uncertain or ambiguous space they would reveal, and sends the best one as a goal.
9. **Outputs.** ROS 2 topics shown live in RViz2, an interactive HTML dashboard, and a versioned JSON twin store.

2. Concepts you must understand

Each entry gives the plain meaning first, then why it matters here.

2.1 Cameras and geometry

Term	Meaning and why it matters
RGB-D camera	Camera that gives colour (RGB) plus depth (D) per pixel, e.g. Intel RealSense. Depth lets us put pixels in 3D without guessing scale.
Intrinsics (K)	Numbers describing the lens: focal length (fx, fy) and image centre (cx, cy). They convert a pixel + depth into a 3D point. For Staer: fx = fy = 672.2, cx = 960, cy = 540 at 1920×1080.
Pose	Position + orientation of the camera, stored as a 4×4 matrix (rotation and translation). "Camera-to-world" pose turns camera points into world points.
Coordinate convention	Different software points the axes differently. Computer vision uses x right, y down, z forward ("optical"); the Staer dataset uses x right, y up, z backward (OpenGL). Converting between them is a flip `diag(1, −1, −1)`. Getting this wrong mirrors the map; we found and fixed exactly this bug early on.
Back-projection	Pixel (u, v) with depth z → 3D point: x = (u − cx)·z/fx, y = (v − cy)·z/fy. Then multiply by the pose to get world coordinates.
IMU	Inertial measurement unit: gyroscope (rotation rate) + accelerometer (acceleration incl. gravity). Helps tracking during fast motion; with a single camera it is what gives metric scale.
Point cloud / voxel	A point cloud is a set of 3D points. A voxel is a small 3D cube (we use 5 cm); storing one averaged point per voxel keeps the map compact.
Occupancy grid	Map of cells marked free, occupied or unknown, built by casting rays from the camera to each depth measurement. Needed to know what was actually observed.
Traversability	2D grid of floor cells a robot can drive on (known free, no obstacle). Used by the planner.

2.2 SLAM (Simultaneous Localisation And Mapping)

Term	Meaning and why it matters
Odometry	Estimating motion between consecutive frames. Like counting steps with eyes closed: each step has a small error, and errors accumulate (drift).
Photometric / geometric alignment	Two ways to compare frames: match brightness of pixels (photometric) or match 3D shapes (geometric, like ICP). "Hybrid" uses both.
Keyframe	A selected frame kept as a reference (we add one every 0.2 m or 12° of motion). New frames are tracked against the keyframe, which reduces drift.
PnP recovery	Fallback when dense tracking fails: match ORB feature points to a keyframe with known 3D positions and solve "Perspective-n-Point" with RANSAC (robust to wrong matches).
Loop closure	Recognising a place visited before. The difference between where we think we are and where we must be reveals the drift.
Pose graph optimisation	All keyframe poses are nodes; measured relative motions (odometry + loops) are edges. An optimiser adjusts all poses at once to agree with every edge as well as possible, spreading the loop correction over the whole path.
ATE (absolute trajectory error)	After aligning the estimated path to the ground-truth path, the root-mean-square position error. Main accuracy metric for SLAM. Lower is better.
RPE (relative pose error)	Error in the motion between nearby frames; measures local drift rather than global error.
Scale	With one camera only, a scene twice as big and twice as far looks identical, so the size of the map is unknown. Depth sensors or an IMU fix this. `ate_sim3_scale` in our tables = how much the path had to be stretched to fit; 1.0 means correct scale.
VIGS-SLAM	A recent visual-inertial SLAM system (camera + IMU) from ETH Zurich that also builds a Gaussian-splatting map. We ran the official code as an external baseline.

2.3 Perception (AI)

Term	Meaning and why it matters
Object detection	Finding objects and drawing a box around each, with a class and a score.
Segmentation	Labelling the exact pixels of each object (a mask), not just a box. Masks give much cleaner 3D shapes than boxes.
Open-vocabulary	The detector is told what to find with words, so new classes need no retraining. Big practical advantage: a warehouse can add "roll cage" tomorrow.
Grounding DINO	Open-vocabulary detector: text prompt + image → boxes. We use the "tiny" model on the GPU.
SAM 2.1	"Segment Anything Model 2.1" from Meta: given a box, returns the object's mask. We use the "tiny" model in bf16 (half precision) for speed.
Confidence / threshold	Detector score between 0 and 1. Detections below the threshold (we test 0.20–0.45) are discarded. Low threshold: more objects found, more false ones.

2.4 Evaluation metrics

Metric	Meaning (higher or lower is better)
Precision	Of the assets in our twin, what fraction are real (matched to a ground-truth asset)? Higher is better.
Recall	Of the real assets that were visible, what fraction did we find? Higher is better.
F1	Harmonic mean of precision and recall; one number that is high only if both are.
Duplicate rate	Extra twin entries per real asset (0 = every asset appears once). Lower is better. This is the main measure of "one ID per asset".
Identity consistency	How consistently the same real asset maps to the same twin ID across all observations (1 = always). Higher is better.
ID switches	How often a real asset's observations jump to a different twin ID. Lower is better.
mIoU	Mean intersection-over-union of 3D voxel labels vs ground truth, averaged over classes. Measures how well labelled the 3D map is.
Map accuracy / completeness	Accuracy: distance from map points to the true surface (lower is better). Completeness: fraction of the true surface covered by the map (higher is better).
Change precision / recall	Of the changes we report, how many are real; of the real changes, how many we report.
Coverage	Fraction of the scene's surface observed during exploration, for a fixed travel distance.

2.5 Software and robotics

Term	Meaning
ROS 2	Robot Operating System 2: standard middleware where programs ("nodes") exchange messages on named channels ("topics"). We use the Jazzy release.
Topic / service	Topic: continuous stream (e.g. `/geosemtwinx/map`). Service: request–reply call (e.g. `/geosemtwinx/save_twin`).
TF	ROS system that tracks coordinate frames; we publish `map → camera_optical`.
RViz2	ROS 3D viewer used for the live demo.
Nav2	ROS navigation stack; our next-best-view goal is published as a standard goal message Nav2 can follow.
WSL2	Windows Subsystem for Linux: real Ubuntu inside Windows, with GPU access. All our code runs there.
Hungarian algorithm	Finds the best one-to-one matching between two sets given a score table (e.g. new observations vs existing assets). Guarantees no asset gets two observations from the same frame.

3. The system, block by block

For each block: **input → what happens → output**, why it is designed that way, and which file implements it (all inside `geosemtwinx/`).

3.1 Data input (`data.py`, `synthetic.py`)

Every dataset is wrapped in the same interface: `seq.K` (intrinsics), `seq.frame(i)` (rgb, depth, optional instance mask), `seq.gt_poses` (ground-truth poses for evaluation only) and `seq.timestamps`. This means every algorithm runs unchanged on any dataset or on the live camera.

Dataset	What it is	Why we use it
TUM RGB-D fr1/desk	Real Kinect recording of an office desk, 573 frames, with motion-capture ground truth	Standard SLAM benchmark; lets us compare with published systems
Staer warehouses	Photorealistic synthetic warehouse; 1920×1080 at 20 fps, depth (16-bit inverse depth), IMU at 400 Hz, instance masks; walkthroughs 001 and 002 of scene03	Real warehouse assets with per-object ground truth to score the twin
Synthetic warehouse	Our own ray-cast scene with racks, pallets, barrels, forklifts; exact asset positions; scripted changes (2 removed, 2 moved, 2 added)	Exact ground truth for fusion, change detection and exploration studies

Staer detail worth knowing

Depth is stored as inverse depth with a per-frame range (`depth_ranges.json`); we decode it back to metres. Videos are decoded with ffmpeg. Instance masks are mapped from part names (e.g. a forklift's wheel) up to the parent asset and its class, so we can score per asset.

3.2 RGB-D SLAM with uncertainty (`slam.py`)

Input: RGB-D frames. **Output:** a pose per frame plus a per-step uncertainty σ.

1. **Tracking:** each frame is aligned to the current keyframe with Open3D's hybrid RGB-D odometry (photometric + geometric terms). A constant-velocity guess is used as the starting point.
2. **Sanity check:** the result is checked by "depth consistency": warp the keyframe depth into the new frame and count how many pixels agree. If tracking failed, PnP recovery (ORB features + RANSAC) is tried and the more consistent answer is kept.
3. **Keyframes:** a new keyframe every 0.2 m or 12°.
4. **Loop closure:** new keyframes are compared with older ones; verified matches add loop edges.
5. **Pose graph:** Open3D optimises all keyframe poses; frames between keyframes follow their keyframe.
6. **Uncertainty:** the odometry's information matrix (how well-constrained the alignment was) gives a per-step σ. This σ is used later by the map and by GASF.

Options for experiments: tracker `hybrid` / `icp` / `hybrid+icp`, and switches to turn off PnP or loop closure (ablations).

3.3 VIGS-SLAM bridge (`vigs_bridge.py`)

To compare with a state-of-the-art visual-inertial system, we export a Staer walkthrough into VIGS-SLAM's input format: PNG images named by nanosecond timestamp, `imu.csv` (EuRoC column order), `calib.txt` (fx fy cx cy) and a config file built from VIGS-SLAM's own `euroc.yaml` with the Staer IMU noise values and the camera-to-IMU transform `Tcb` computed from the dataset's `sensor.yaml` files. After VIGS-SLAM runs in its own conda environment, we read `traj_full_afterBA.txt` and interpolate it to our frame times (linear for position, slerp for rotation), so VIGS-SLAM poses can drive the same mapping and fusion code.

3.4 Perception (`detectors.py`)

Input: an RGB keyframe and the class list. **Output:** a list of detections, each with label, score, box and mask.

Grounding DINO receives a prompt like `"forklift. pallet. cardboard box. pallet truck."` and returns boxes above the threshold. SAM 2.1 then segments each box. Both run on the GPU; SAM runs in bf16. Model checkpoints download automatically the first time. Detection runs only on keyframes (every 3rd frame by default) because it is the most expensive step (354 ms per keyframe).



Figure 2. Staer frame 300: Grounding DINO boxes with scores, SAM 2.1 masks tinted by class, and persistent twin IDs projected back into the image ("A10 forklift", "A12 cardboard box").

3.5 Lifting detections to 3D (`gasf.py` : `make_segments`)

Each mask's pixels are back-projected with depth and the camera pose, giving a 3D point cloud. Cleaning steps: keep the largest connected component near the camera (removes background leaking through the mask), drop points beyond the depth limit, and for floor-standing classes (forklift, pallet truck) require contact with the floor so floating false detections are rejected. Each segment gets a descriptor: centroid, robust extent (2nd–98th percentile, so a few stray points do not inflate the box), voxel set, and size.

3.6 GASF: Geometry-Aware Semantic Fusion (`gasf.py`) — the core contribution

Input: 3D segments from one keyframe. **Output:** an updated set of persistent assets.

Step 1: score every (segment, asset) pair.

$$S = 0.25 \cdot \text{sem} + 0.35 \cdot \text{overlap} + 0.20 \cdot \text{centroid} + 0.20 \cdot \text{geo}$$

- **sem**: do the class labels agree (weighted by confidence)?
- **overlap**: fraction of the segment's voxels that touch the asset's voxels (slightly dilated to tolerate drift).
- **centroid**: closeness of centres, scaled by the object size and pose uncertainty.
- **geo**: similarity of 3D size/shape.

Geometry carries 75% of the weight. A different label can still match if the geometry is very convincing (overlap ≥ 0.5 and shape ≥ 0.5): this fixes cases where the detector calls the same pallet truck "pallet" in one frame.

Step 2: match. The Hungarian algorithm picks the best one-to-one matching; pairs below a minimum score start new candidates.

Step 3: confirm. A candidate becomes a real asset only after ≥ 2 hits from ≥ 2 different viewpoints (the "multi-view gate"). This removes one-off false detections.

Step 4: update. Points are added, the label distribution is updated with confidence weights (a 0.9 detection counts more than a 0.3 one), and the robust extent is recomputed.

Step 5: prune and merge. If an asset should have been visible (in view, not occluded, within range) but was not detected, it gets a "miss"; repeated misses remove unconfirmed candidates. Fragments of the same object (e.g. two halves of a forklift) are merged when they touch and their union has a plausible size.

> **Why this beats the baselines**
> **Per-frame labelling** keeps every detection: huge duplicates. **Centroid tracking** matches by centre distance only: works with perfect poses, but with drift two sightings of one object drift apart and become two assets. **Geometry-only clustering** ignores labels: neighbouring objects of different classes get merged. GASF combines all cues, plus the multi-view gate and merging.

3.7 Uncertainty-aware mapping (`mapping.py`)

Every depth frame is back-projected and fused into a 5 cm voxel map. Each measurement gets a weight:

$$w = 1 \,/\, \sigma_z(z)^2 \cdot \exp(-\sigma_{\text{pose}} \,/\, 0.1)$$

- **$\sigma_z(z)$**: depth noise grows roughly with distance squared, so far points count less.

- σ_{pose}: if the camera position for this frame is uncertain, the whole frame counts less; above 0.15 m the frame is skipped entirely ("gated").
- Measurements whose own noise is above a limit are cut; a voxel needs ≥ 2 observations to appear.

The weighted average per voxel gives a clean surface. In parallel, a log-odds **occupancy map** records free / occupied / unknown space by ray casting, and a **traversability grid** marks drivable floor.

> **Implementation note**
> We first used Open3D's built-in TSDF volume, but with Open3D 0.20 on Python 3.13 it returned empty maps. We replaced it with our own NumPy fusion (`FusedMap`), which also made the uncertainty weighting explicit.

3.8 Digital twin (`twin.py` , `dashboard.py`)

Confirmed assets are committed to a `DigitalTwin` : each asset has an ID (A0001…), class, confidence, median centroid, robust 3D box, number of observations, first/last seen. Every session is saved as a new version in the **twin store** (`twin_v001.json` , `twin_v002.json` , plus an index). Calling `update()` with a new session keeps IDs of assets that are still there, so an asset keeps its identity across days. Spatial queries ("assets of class X within r metres of (x, y)") are supported.

The **HTML dashboard** is one self-contained file (no server needed) with four tabs: Digital twin (3D view, asset inspector, spatial query, filters), Changes & versions, Comparative studies, System performance.

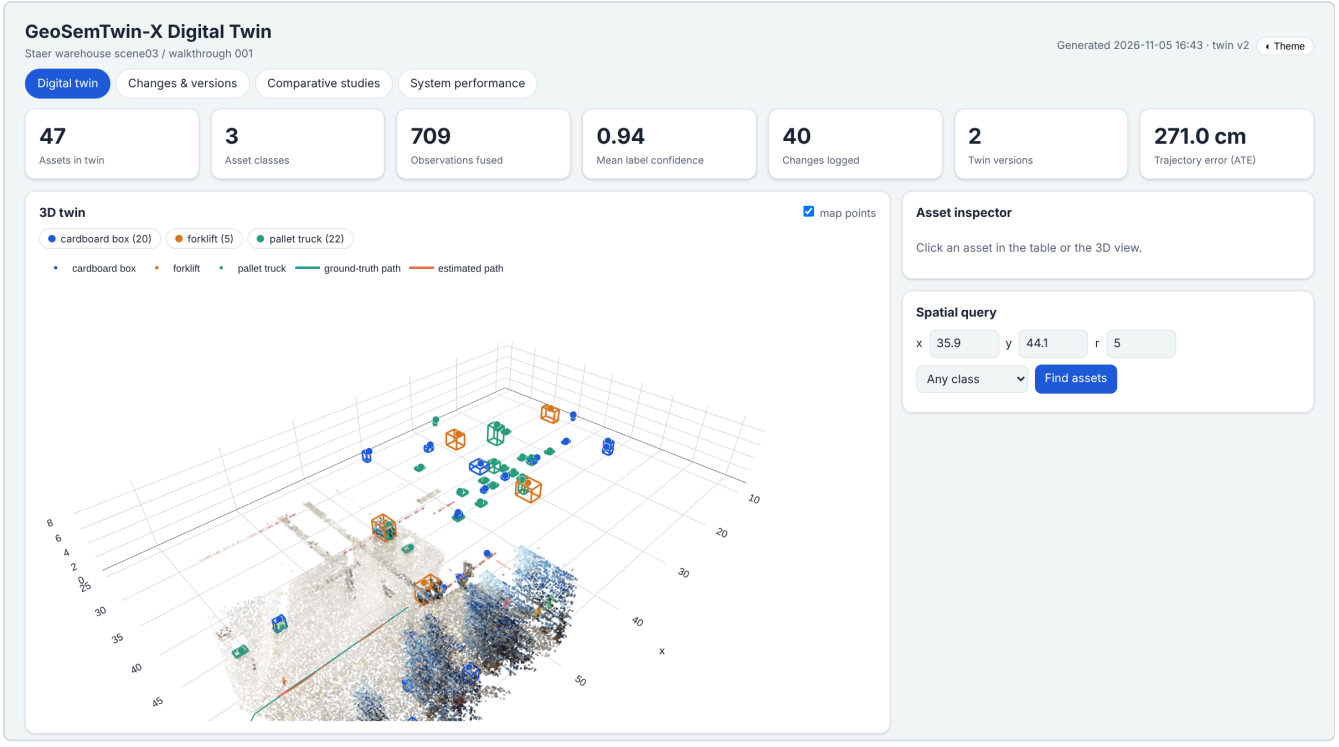


Figure 3. Main dashboard (Staer): KPI cards, interactive 3D twin with asset boxes by class, map points, ground-truth and estimated paths, inspector and spatial query.

3.9 Verified change detection (`change.py`)

Old and new versions are matched with the Hungarian algorithm (within a gate of 4σ). Then each candidate change is **verified against the occupancy maps**:

- **Moved**: matched but the centre shifted > 0.5 m, and occupancy confirms the object at the new place in the new session and at the old place in the old session.
- **Removed**: unmatched old asset, and its old location was *observed free* in the new session.
- **Added**: unmatched new asset whose location was *observed free* in the old session.
- Otherwise: **unverified** (we did not look there well enough to say). Unverified items are logged but not counted as changes.

This is why the second Staer walkthrough shows many "unverified" entries in the dashboard and 0 false alarms: the system honestly says "not observed", instead of "removed".

3.10 Next-best-view exploration (`nbv.py`)

$$\mathrm{U(view)} = (\mathrm{w_u}\cdot\mathrm{unknown} + \mathrm{w_s}\cdot\mathrm{uncertain} + \mathrm{w_a}\cdot\mathrm{ambiguous}) \cdot \mathrm{e}^{-0.15\cdot\mathrm{d}}$$

Candidate positions on drivable floor are scored by how many **unknown** cells, **uncertain** surface cells and **ambiguous** assets (unconfirmed or mixed label) they would see, discounted by travel distance d. The best one is reached with A* path planning on the traversability grid; a simulated multi-ring LiDAR (heights 0.08–1.0 m) checks for low obstacles and triggers replanning. In the live system the goal is published on `/geosemtwinx/nbv_goal`.

3.11 Live operation and ROS 2 (`streaming.py`, `ros2/`)

`LiveTwin` runs the whole pipeline frame by frame. The ROS 2 node `geosemtwinx_node.py` subscribes to synchronised RGB, depth and camera-info topics, runs LiveTwin in a worker thread (always processing the newest frame), and publishes pose, path, map, assets (boxes + ID labels), traversability, NBV goal and path, an annotated image and a status string, plus the TF `map → camera_optical`. `dataset_player.py` replays Staer or TUM as ROS topics, so the same node works with recorded data or a real RealSense camera. `run_demo.sh` starts player + node + RViz2 in one command.

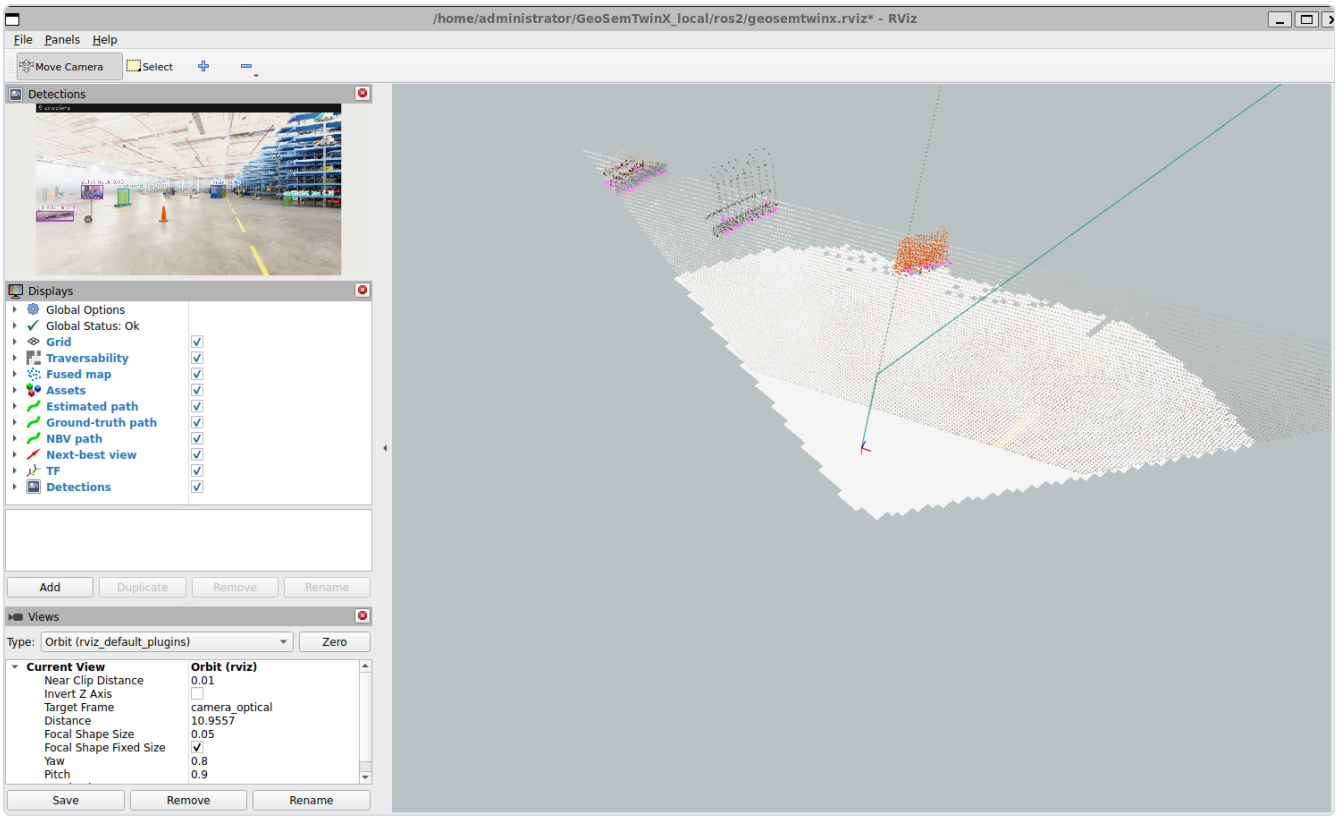


Figure 4. The live ROS 2 demo in RViz2: detection image (top-left), fused map, traversability (white floor), asset clusters, TF axes and paths.

4. What was actually done: the implementation journey

Judges like to hear that you understand the engineering, including what went wrong. This is the real sequence.

Stage 1: Colab notebook and the camera-convention bug

The first version ran in Google Colab. Interpreting its output, we found the Staer maps were mirrored: the code used the flip `diag(1, −1, 1)` instead of `diag(1, −1, −1)`. We verified the correct convention by re-projecting depth between frames with each candidate flip and choosing the one with the lowest error, and confirmed that Staer depth is planar (z-distance), not ray length.

Stage 2: Building the full prototype package

We wrote the `geosemtwinx` Python package (about 4,100 lines across 20 modules, plus about 600 lines of scripts and ROS 2 nodes) plus the experiment runner, ROS 2 nodes and dashboards. Several algorithmic fixes came from testing on synthetic data:

- SLAM: tracking selection by depth consistency; correct pose-graph edge direction; final keyframe so loops close at the end. Synthetic ATE improved from 37 cm to 19 cm.
- Mapping gate: initially gated by accumulated σ (which rejected everything after a while); switched to per-step σ.
- Open3D TSDF returned empty maps → replaced with our own `FusedMap`.
- Change detection: precision was only 0.56 → added occupancy verification → 1.00.
- Exploration: frontier baseline got stuck → added blacklisting; robot hit low obstacles → added a 0.08 m LiDAR ring; live planner trapped by drift "ghost walls" → clear the swept path and a blind zone.
- Evaluation: counted only assets that were really detectable (≤ 8 m, ≥ 0.2% of the image, seen in ≥ 2 keyframes).

Stage 3: Setting up the GPU laptop

- WSL2 had two Ubuntus: ROS 2 Jazzy lived in **Ubuntu-24.04**, VIGS-SLAM's conda env (`vigs-slam`) in the other. We moved the project to Ubuntu-24.04 and exchanged files with the VIGS-SLAM side through the D: drive.
- `setup.sh` created a Python virtual environment that can also see ROS 2 (`--system-site-packages`), installed PyTorch for CUDA 12.6 and the requirements. `check` confirmed CUDA, the RTX 3500 Ada (12 GB, bf16), ffmpeg, the Hugging Face login and rclpy.
- Model download crashed with `process() takes no keyword arguments`: Ubuntu's old `brotli` package leaked into the environment. Fixed with `pip install -U brotli zstandard`.

Stage 4: Running experiments and diagnosing Staer SLAM

TUM gave 2.9 cm ATE. On Staer, both our SLAM (2.7 m) and VIGS-SLAM (5.0 m) failed. Instead of guessing, we tested hypotheses one at a time:

1. Frame misalignment between video streams? All streams had exactly 361 frames, matching 361 timestamps → ruled out.

2. Wrong intrinsics or depth scale? We warped frames using ground-truth poses with scaled focal length and scaled depth; the unscaled values (1.0) gave the lowest error → ruled out.
3. Motion: the ground truth shows the camera moves 5.0 cm per frame, constant, with essentially zero rotation (median 0.00°, max 0.95°). A straight constant-speed pass has no acceleration, so a camera+IMU system cannot observe scale; and a long, low-texture aisle with median depth 8 m is a known weak case for RGB-D tracking. **Cause found.**

How to say it

"We found that this walkthrough is a degenerate trajectory for camera-based SLAM: straight and at constant speed. We proved the data and calibration were correct, and showed that both our SLAM and the state-of-the-art VIGS-SLAM lose scale for the same physical reason. That is exactly why our roadmap adds LiDAR."

Stage 5: Demo, live ROS 2, dashboards and deck

We produced an offline demo video (camera view + detections + persistent IDs on the left, live top-down twin with next-best view on the right), ran the live ROS 2 demo in RViz2, generated dashboards for TUM, Staer and the live run, and built the presentation from the real result files.

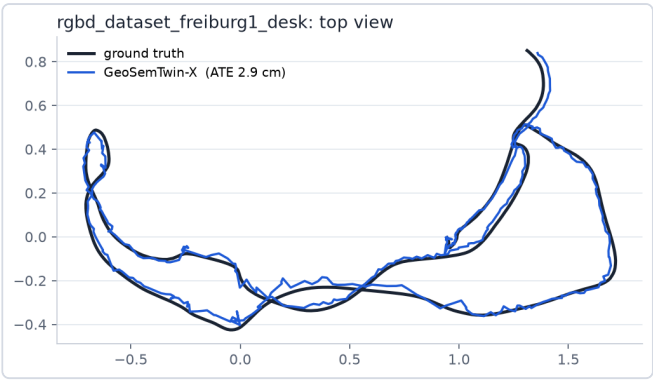
5. Experiments and results: how to read every number

For each study: the question, how it was measured, the result, and what to say.

5.1 SLAM accuracy on TUM fr1/desk

Question: is our localisation accurate enough to build a twin? **How:** ATE after aligning to motion-capture ground truth; ablations remove one component at a time; published numbers for well-known systems for context.

Method	ATE (cm)
GeoSemTwin-X (full)	**2.9**
w/o PnP recovery	3.0
w/o loop closure	5.4
Odometry only	7.3
ORB-SLAM2 (published)	1.6
ElasticFusion (published)	2.0
DVO-SLAM (published)	2.1



Estimated (blue) vs ground truth (black).

What to say

"Loop closure and pose-graph optimisation cut the error by 60%, from 7.3 to 2.9 cm. That is within about one centimetre of ElasticFusion and DVO-SLAM. ORB-SLAM2 is more accurate; our SLAM is a lightweight front end that also produces the uncertainty our twin needs, and the architecture can use VIGS-SLAM or any better tracker."

5.2 SLAM on Staer and VIGS-SLAM

Method	ATE (m)	Scale needed	Drift
GeoSemTwin-X RGB-D SLAM	2.71	1.30	15%
VIGS-SLAM (mono + IMU)	4.98	2.30	28%

Both fail on this sequence for the physical reason explained in Chapter 4 (degenerate motion). Our RGB-D SLAM is the better of the two here because depth gives it partial scale. A configuration sweep on the first 6 s found that adding ICP (geometry) helped most (0.84 m vs 1.43 m), supporting the plan to add LiDAR.

Do not

present the Staer ATE as "our SLAM accuracy". Present it as a finding about camera-only SLAM on straight warehouse aisles, and use TUM for the accuracy claim.

5.3 Semantic fusion: GASF vs conventional methods (the main study)

Question: does GASF produce one stable twin entry per real asset? **How:** the same detections are fused by four methods; scored against exact ground truth (synthetic, 22 assets, mean of 3 seeds, SLAM poses).

Method	Objects	Precision	Recall	F1	Dup./asset	ID switches	mIoU
Independent 2D labelling	289	0.08	1.00	0.14	10.60	237	0.28
Geometry-only clustering	21	0.55	0.52	0.53	0.46	7	0.20
Centroid tracking	29	0.71	0.94	0.81	0.29	33	0.33
GASF (ours)	**22**	**0.81**	**0.82**	**0.81**	**0.11**	**9**	**0.35**

How to read it: per-frame labelling finds everything (recall 1.00) but reports 289 objects for 22 assets, so precision is tiny. Centroid tracking and GASF have the same F1, but GASF has **2.5× fewer duplicates**, **3.7× fewer ID switches** and higher precision; it trades a little recall for a cleaner registry. GASF's twin has 22 entries for 22 assets. Its mIoU is also the best.

Ablation: which parts of GASF matter?

Variant	F1	Effect
Full GASF	**0.81**	—
w/o duplicate merge	0.76	duplicates 0.11 → 0.45
w/o multi-view gate	0.59	false positives 2.3 → 16.7 per run
w/o geometry terms	0.34	ID switches 9 → 58

What to say

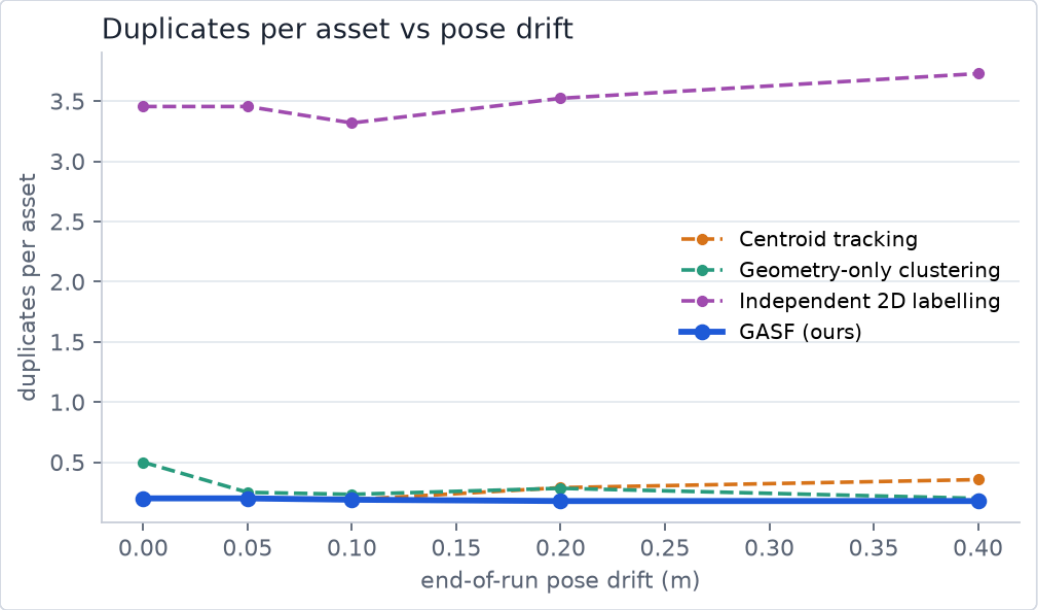
"Every component earns its place. Removing geometry collapses F1 from 0.81 to 0.34, which is direct evidence for our main idea: geometry must decide identity."

On the real Staer warehouse

With ground-truth poses and the default threshold (0.30), GASF and centroid tracking tie (perfect poses are centroid tracking's best case). At the better threshold 0.20, GASF wins: F1 0.30 vs 0.29, duplicates 0.08 vs 0.31, identity consistency 0.85 vs 0.69. Per-frame labelling: 4.72 duplicates per asset. Recall is low (19%) because the general-purpose detector misses many small or distant assets; that is a detector limitation, not a fusion one.

Note on Staer ablations: there, removing the multi-view gate *raised* F1 (0.23 vs 0.12) because detections are sparse (few assets are seen twice). Be ready to explain: the gate trades recall for precision; with a stronger detector or denser keyframes the synthetic result applies.

5.4 Robustness to localisation drift



Staer: duplicates per asset as artificial pose drift grows from 0 to 40 cm.

At 40 cm drift, GASF stays at 0.18 duplicates per asset while centroid tracking doubles to 0.36 and per-frame labelling is at 3.7. **Say:** "GASF's advantage grows exactly when localisation gets worse, which is the realistic case."

5.5 Map quality (uncertainty-aware fusion)

Staer, ground-truth poses	Ours	Naive average
Points within 5 cm of true surface	99.3%	91.5%
Median / 90th-percentile error	1.0 / 2.1 cm	1.6 / 4.7 cm
Map points	179 k	402 k
Completeness	89%	100%

Read: our map is cleaner and less than half the size; the price is 11% lower completeness because unreliable far measurements are dropped. Always mention this trade-off: it shows you understand the method.

5.6 Change detection

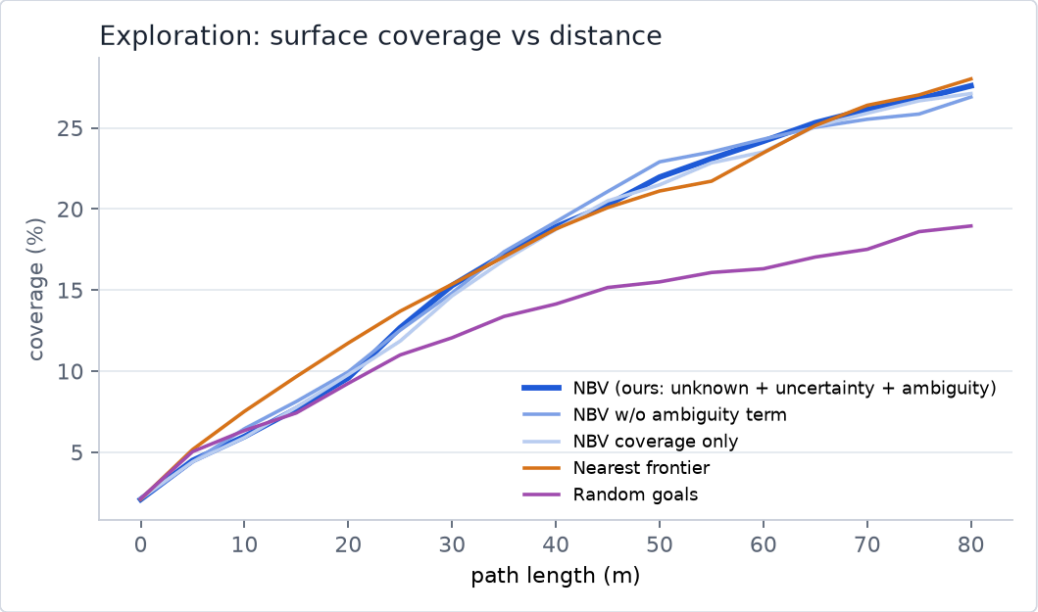
Synthetic (6 scripted changes)	Precision	Recall	F1
With occupancy verification (ours)	**1.00**	**0.83**	**0.91**
Without verification	0.56	0.83	0.67

Verification removes every false alarm without losing any true change. On the second Staer walkthrough (nothing actually changed), the system reported 0 false changes and marked unobserved assets as "unverified".

5.7 Exploration (next-best view)

80 m travel budget, 3 seeds	Coverage	Assets identified	Collisions
NBV (ours)	**27.6%**	**94%**	**1.0**
Nearest frontier	28.0%	96%	0.0
Random goals	19.0%	71%	0.7

Honest reading: NBV matches the classic frontier planner and clearly beats random goals; in this test the semantic ambiguity term did not add measurable benefit. Claim "on par with frontier exploration, with uncertainty-driven goals", not "better".



Coverage vs distance travelled.

5.8 Speed, power and memory

Stage	Time
Grounding DINO + SAM 2.1 (GPU), per keyframe	354 ms
RGB-D SLAM tracking (CPU), per frame	1072 ms
Map integration, per frame	46 ms
Occupancy update / 3D lifting / GASF, per keyframe	19 / 14 / 3 ms

End to end 0.8 FPS; GPU 74 W average while detecting (27 W idle, 130 W peak), 75% utilisation, 2.9 GB peak GPU memory, 7.3 GB RAM, about 26 J per keyframe. **Read:** GASF itself is almost free; the bottleneck is CPU SLAM, which is why the roadmap moves tracking to the GPU or to VIGS-SLAM.

6. Code map and how to run everything

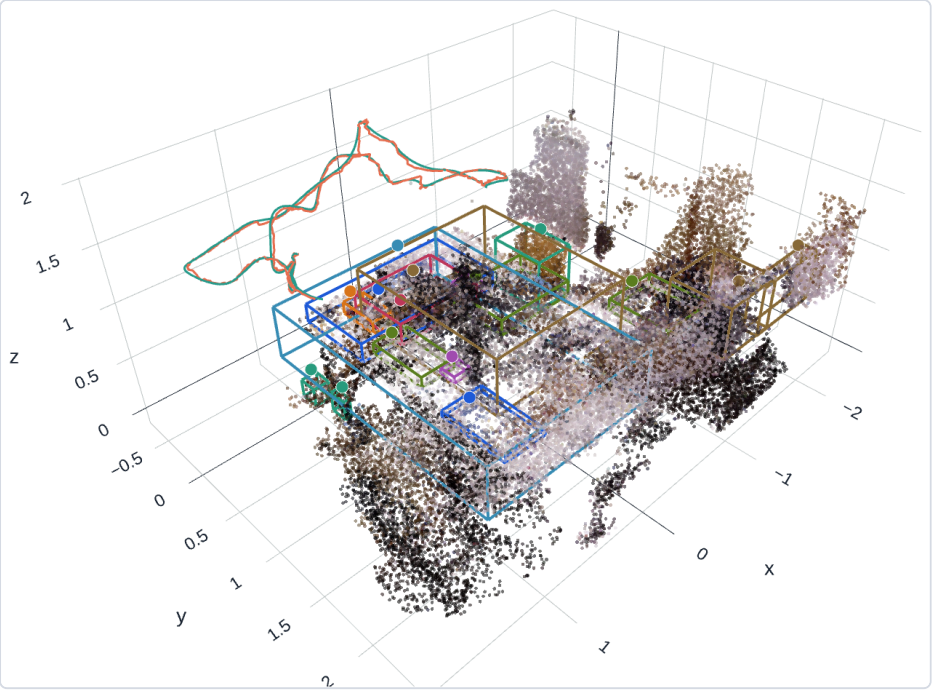
File	Purpose
`geosemtwinx/data.py`	Dataset readers (TUM, Staer, synthetic) with one common interface
`geosemtwinx/synthetic.py`	Ray-cast synthetic warehouse, trajectories, scripted changes
`geosemtwinx/slam.py`	RGB-D SLAM: odometry, PnP recovery, keyframes, loop closure, pose graph, σ
`geosemtwinx/vigs_bridge.py`	Export to VIGS-SLAM, run it, load its trajectory
`geosemtwinx/detectors.py`	Grounding DINO + SAM 2.1; simulated noisy detector for tests
`geosemtwinx/gasf.py`	3D lifting, GASF fusion, baselines
`geosemtwinx/mapping.py`	Uncertainty-weighted voxel map, occupancy, traversability
`geosemtwinx/change.py`	Verified change detection and scoring
`geosemtwinx/twin.py`	Digital twin, versions, queries, twin store
`geosemtwinx/nbv.py`	Next-best-view planner and exploration simulator
`geosemtwinx/evaluation.py`	Precision/recall/F1, duplicates, identity, mIoU
`geosemtwinx/compare.py`	All comparative studies, ablations, plots, tables
`geosemtwinx/streaming.py`, `demo.py`	Live pipeline (LiveTwin) and offline demo video
`geosemtwinx/dashboard.py`, `power.py`	HTML dashboard; GPU power monitor (nvidia-smi)
`scripts/run_all.py`	One entry point for every experiment
`ros2/`	ROS 2 node, dataset player, RViz config, `run_demo.sh`
`configs/`	Parameters for Staer, TUM and RealSense

Commands (in Ubuntu-24.04, project folder)

```
source /opt/ros/jazzy/setup.bash
source .venv/bin/activate
python scripts/run_all.py check --vigs-root "/mnt/d/Gayathri/Robot Navigation/VIGS-SLAM"
python scripts/run_all.py tum                # SLAM study + office twin           (~20 min)
python scripts/run_all.py vigs-export --vigs-root "/mnt/d/Gayathri/Robot Navigation/VIGS-SLAM"
#   then run VIGS-SLAM in the other Ubuntu (see RUN_GUIDE.md)
python scripts/run_all.py staer --change-walk 002 --vigs-traj /mnt/d/Gayathri/vigs_run/out
(~30 min)
python scripts/run_all.py synthetic          # fusion, change, exploration        (~50 min)
python scripts/run_all.py demo               # demo video                         (~10 min)
python scripts/run_all.py report             # results/REPORT.md
bash ros2/run_demo.sh staer                  # live ROS 2 demo in RViz2
explorer.exe results/GeoSemTwinX_Dashboard.html
```

Where the results are

`results/REPORT.md` (all tables), `results/GeoSemTwinX_Dashboard.html`, `TUM_Dashboard.html`, `ros_staer/Live_Dashboard.html`, `demo_staer/demo.mp4`, `figures/` (charts), `comparative/` (CSV tables), `twin_store/` (versions), `power.json`.



TUM office twin: 16 assets of 8 classes with boxes, fused map and both trajectories.

7. How to present

7.1 The story in three sentences

"Warehouses need a live inventory of where every asset is, but AI detectors work frame by frame and count the same forklift hundreds of times, and localisation drift makes it worse. GeoSemTwin-X lets 3D geometry decide identity and tracks uncertainty end to end, so every physical asset gets one stable ID and changes are reported only when the sensors actually confirm them. On real and synthetic warehouses this gives 2.5× fewer duplicates and 3.7× fewer identity switches than the best conventional method, perfect change-detection precision, and it runs on one laptop GPU with ROS 2."

7.2 Slide-by-slide script (about 8 minutes)

Slide	What to say (key points)	Time
1 Cover	Name the project and the one-line idea: geometry-grounded semantic digital twin.	15 s
2 Challenge	Twins go stale; per-frame AI gives 289 objects for 22 assets; drift splits assets; robots need to know where to look.	45 s
3 Architecture	Walk the pipeline left to right once; then read the five numbers at the bottom.	75 s
4 Pros / cons	Point to the GASF column; mention open-vocabulary and verified change; state limits briefly.	45 s
5 Technical	Explain the GASF score (75% geometry) and the uncertainty weight; one sentence each for change and NBV.	60 s
6 Plan	Phase 1 done; LiDAR and edge next; tech stack is fully open source.	30 s
7 Prototype	Show RViz (or play the demo video); ROS 2 topics; one command; 0.8 FPS, 74 W.	45 s
8 Perception + twin	Detection + masks with persistent IDs; 3D maps; dashboard.	40 s
9 SLAM	2.9 cm on TUM, loop closure −60%; Staer finding motivates LiDAR.	40 s
10 Fusion study	The headline comparison and the ablation (geometry 0.34 → 0.81).	50 s
11 Robustness etc.	Drift plot, map 99.3%, change precision 1.00, NBV ≈ frontier.	45 s
12–14	Cost < ₹5 lakh for a pilot, assumptions, references (brief).	30 s

7.3 Numbers to memorise

2.9 cm TUM ATE (−60% with loop closure)	**2.5× / 3.7×** fewer duplicates / ID switches vs centroid	**0.81 → 0.34** F1 without geometry terms	**0.56 → 1.00** change precision with verification
99.3% map points within 5 cm	**0.18 vs 0.36** duplicates at 40 cm drift (GASF vs centroid)	**3 ms** GASF per keyframe	**74 W · 2.9 GB** GPU power · GPU memory

7.4 Live demo tips

- Record a backup video of `bash ros2/run_demo.sh staer` beforehand (Win+G); play the video if anything fails.
- In RViz, zoom out, untick Grid, set Fused map point size to 0.05; point out an asset box appearing and keeping its ID.
- Open `GeoSemTwinX_Dashboard.html`: rotate the 3D twin, click an asset in the table, run a spatial query, open the Changes tab and explain "unverified".
- The demo plays recorded data at about 0.5–0.8 FPS; say so before the judges ask.

7.5 Likely judge questions and good answers

Q1. What is new here compared with existing semantic SLAM?

Most systems attach labels to a map or track objects by centre position. We make identity a geometry decision with an explicit score, gate it by multiple views, merge fragments, carry pose uncertainty into the map and the matching, and verify changes against occupancy. The result is a versioned asset registry, not just a labelled map.

Q2. Why not just use a better detector?

A better detector improves recall, but it still works one frame at a time: the duplicate problem remains. Fusion is needed regardless. Our per-frame baseline uses the same detector and gives 10.6 duplicates per asset.

Q3. Centroid tracking has the same F1. Why is GASF better?

F1 hides registry quality. GASF has 2.5× fewer duplicates, 3.7× fewer ID switches and higher precision, and the gap widens with drift (0.18 vs 0.36 duplicates at 40 cm). For inventory, a duplicate is a phantom asset, which is costly.

Q4. Your Staer SLAM error is 2.7 m. Doesn't that break everything?

That sequence is a straight, constant-speed pass: a known degenerate case for camera-based SLAM. We verified the data and calibration, and VIGS-SLAM fails the same way. Our twin results on Staer use ground-truth poses plus injected drift up to 40 cm, which isolates the fusion method; LiDAR fusion is the planned fix for localisation.

Q5. Why is recall only 19% on Staer?

The open-vocabulary detector (tiny model, no training) misses small, distant or rack-mounted boxes in 1920×1080 frames scaled down by half. It is a detector limitation, not fusion. Larger models or a few hundred labelled site images would raise it; the fusion layer does not change.

Q6. Is it real-time?

Not yet: 0.8 FPS end to end, limited by CPU SLAM (1.07 s/frame). Detection is 354 ms per keyframe and GASF only 3 ms. Moving tracking to the GPU or to VIGS-SLAM, and detecting on fewer keyframes, are the next steps. Inventory twins do not need video-rate updates; a robot moving slowly is sufficient.

Q7. How do you know a change is real?

A change is reported only if the occupancy map confirms it: for "removed", the old location must have been observed empty in the new session. Otherwise it is "unverified". Precision went from 0.56 to 1.00 with this check.

Q8. Why not evaluate on KITTI or ScanNet as in the abstract?

KITTI and SemanticKITTI are LiDAR driving datasets; they need a LiDAR SLAM front end, which is Phase 2. ScanNet needs a signed licence. We used TUM (SLAM benchmark) and the Staer warehouse with per-object ground truth, which match the warehouse use case more closely.

Q9. How does the twin keep the same ID across days?

The new session's assets are matched to the previous version (Hungarian + 4σ gate); matched assets keep their IDs, and the twin store records each version with a timestamp.

Q10. What does it cost to deploy?

The prototype needs no extra cost: open-source software, no training data. A pilot robot with RGB-D camera, LiDAR, mobile base and Jetson edge computer is indicatively under ₹5 lakh.

Q11. What happens with moving forklifts and people?

Assets are confirmed only after multiple views; a moving object produces inconsistent geometry and is less likely to be confirmed, and change detection compares sessions, not frames. Dynamic-object handling is listed as future work.

Q12. Why are the GASF weights 0.25 / 0.35 / 0.20 / 0.20?

Chosen so geometry dominates (75%), with overlap as the strongest single cue because it is most robust to label noise. The ablation shows geometry is essential; tuning the exact weights is part of future work.

Q13. Where does the uncertainty come from?

From the SLAM's information matrix (pose σ per step) and a depth-noise model that grows with distance. Both enter the voxel weights and the matching gate.

Q14. What is next-best view adding if it equals frontier?

It reaches the same coverage while choosing goals that also reduce uncertainty and resolve ambiguous assets; with a stronger semantic signal (real detector ambiguity) we expect the ambiguity term to matter more. We report the current result honestly.

8. Appendix: limitations and how to talk about them

Limitation	Honest framing	Next step
Not real-time (0.8 FPS)	Prototype with CPU SLAM; perception and fusion are already fast	GPU tracking / VIGS-SLAM; fewer keyframes
Staer SLAM fails on straight aisle	Degenerate motion for any camera SLAM; diagnosed and proven	LiDAR / wheel odometry fusion
Low detector recall on Staer	Open-vocabulary tiny model, no training	Larger or fine-tuned detector
NBV ≈ frontier	Equal coverage, adds uncertainty-driven goals	Real ambiguity signal, longer runs
Map completeness 89%	Deliberate: unreliable far depth is dropped	Revisit regions flagged uncertain
No KITTI / ScanNet	Needs LiDAR SLAM / licence	Phase 2
Simulated LiDAR, recorded data	ROS 2 interface ready for a real camera (RealSense config included)	Real robot trials

General rule

State the number, give the physical or engineering reason, and name the fix. Judges trust a team that knows exactly where its system fails.

Summary checklist before the presentation

- Fill PID, team names and photos on the cover; rename the file with your PID.
- Replace indicative costs with vendor quotes.
- Record the RViz demo and keep the dashboard HTML open in a browser tab.
- Practise the three-sentence story and the eight numbers.
- Re-read Q1–Q5; they are the most likely.